Short-Term Solar Power Forecasting

Comparing FTDNN, NARX, and LSTM

Final Report

Evan Burk

Deep Learning Final Project

Oklahoma State University

May 2026

Abstract

This report presents a comparative study of short-term solar power forecasting using three sequence-model families: Focused Time Delay Neural Networks (FTDNN), Nonlinear Autoregressive Networks with Exogenous Inputs (NARX), and Long Short-Term Memory networks (LSTM). The forecasting task uses self-collected laboratory data containing solar-panel, battery, weather, irradiance, and seasonal time features. Daily log files are merged into a chronological time series, aligned to a one-minute grid, and converted into supervised windows that use 120 minutes of history to predict PV power 60 minutes ahead. Both FTDNN and NARX are evaluated in linear and nonlinear forms, while LSTM serves as the recurrent benchmark. In the final run, the best-performing model was the nonlinear FTDNN with MAE = 22.449 W, RMSE = 31.218 W, and R^2 = 0.5452. The study shows that, for this one-hour horizon, a simpler nonlinear delayed-input model can outperform a more complex recurrent network on real laboratory data.

1. Introduction

Accurate short-term solar forecasting is useful for renewable-energy integration, battery charging decisions, and operational planning. Because photovoltaic output can change quickly with irradiance and local weather conditions, a one-hour forecast horizon is long enough to be practically useful while still short enough for recent measurements to contain predictive signal.

The original proposal for this project focused on LSTM-based forecasting in PyTorch using self-collected data from the solar laboratory. During development, the study was expanded to include FTDNN and NARX baselines, including linear variants, so the final comparison would show whether a more complex recurrent model actually outperformed simpler delayed-input and autoregressive approaches.

The rest of this report is organized as follows. Section 2 summarizes the project schedule and milestones. Section 3 describes the dataset. Section 4 explains the selected network families and the training approach. Section 5 describes the experimental setup and implementation details. Section 6 presents results and discussion. Section 7 gives summary conclusions. Section 8 lists references, and Appendix A summarizes the submitted code.

2. Project Schedule

The course project brief required a proposal, a presentation, and a final report, with proposal submission due on March 26, 2026, presentation slides due on April 27, 2026, oral presentations on April 28 or April 30, and the final report due on May 5, 2026. The proposal also outlined a six-week development plan. Since exact per-day GitHub milestone timestamps are not reproduced in this report, the schedule below summarizes the project phases that guided the work.

- Week 1: finalize data preprocessing, confirm the persistence baseline, and verify the daily log-ingestion pipeline.
- Week 2: implement the initial LSTM model proposed at the start of the project.
- Week 3: Expand the system to include FTDNN and NARX, in both linear and nonlinear forms.
- Week 4: train and tune the models, check validation behavior, and refine the windowing and residual-prediction approach.
- Week 5: generate comparison plots, prepare the presentation, and interpret why the best-performing model won.
- Week 6: write the final report, organize repository deliverables, and prepare documented code for submission.

This schedule reflects the final direction of the project: rather than presenting only an LSTM forecast, the completed work becomes a comparative study of feedforward, autoregressive, and recurrent sequence models on the same solar forecasting task.

3. Description of the Data Set

The dataset is self-collected from the solar laboratory and stored as daily CSV files named gpg_YYYY-MM-DD.csv. The logs include timestamps, panel electrical measurements, battery measurements, environmental variables, solar irradiance, UV index, and a derived efficiency estimate. This is a real

operational dataset rather than a standardized benchmark, which makes the forecasting results more directly tied to the lab system.

In the final run, preprocessing produced 28,173 one-minute rows and 19 input features. The features can be grouped into three categories: (1) electrical and battery signals such as PV voltage, PV current, battery voltage, battery current, battery power, and average battery power; (2) environmental variables such as battery temperature, air temperature, humidity, air pressure, wind speed, wind direction, solar irradiance, and UV index; and (3) derived or seasonal variables such as estimated efficiency, minute-of-day sine and cosine terms, and day-of-year sine and cosine terms.

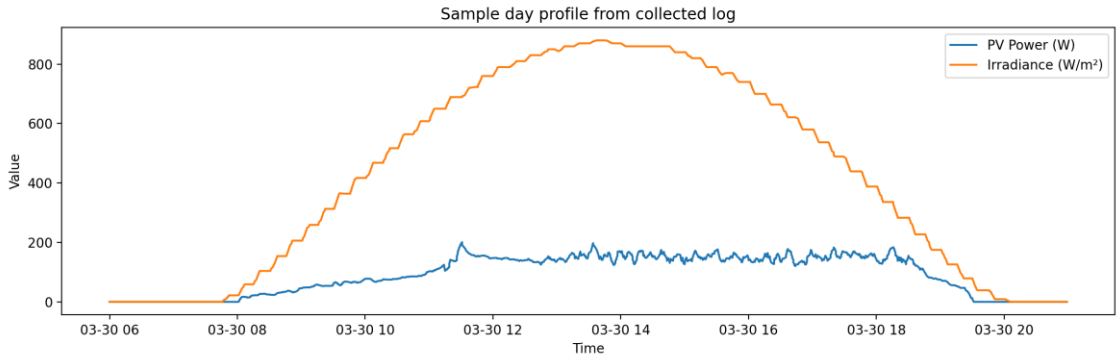

Figure 1. Sample day profile from a collected daily log. PV power follows the daylight cycle but also shows short-term variation relative to irradiance.

The time-feature engineering step uses cyclic encodings instead of raw clock values. Minute-of-day is transformed into mod_sin and mod_cos, and day-of-year is transformed into doy_sin and doy_cos. This avoids artificial discontinuities at midnight and at the end of the year, allowing the models to interpret time as a repeating cycle rather than as a strictly increasing number.

4. Deep Learning Networks and Training Algorithm

Three model families were used in this study. FTDNN is a feedforward sequence model that applies a tapped delay line to the input and then uses a static multilayer network. NARX is a nonlinear autoregressive model with exogenous inputs, meaning it uses both recent external signals and recent output history. LSTM is a recurrent model with gated memory designed to preserve long-range temporal information while maintaining stable training.

4.1 FTDNN

The FTDNN configuration converts the previous two hours of exogenous input history into a flattened feature vector. In the linear baseline, the model is a single linear layer. In the nonlinear version, two hidden layers with ReLU activation and dropout are added before the final output layer. This architecture is well matched to short-horizon forecasting because it can exploit finite recent history without the training complexity of recurrent feedback.

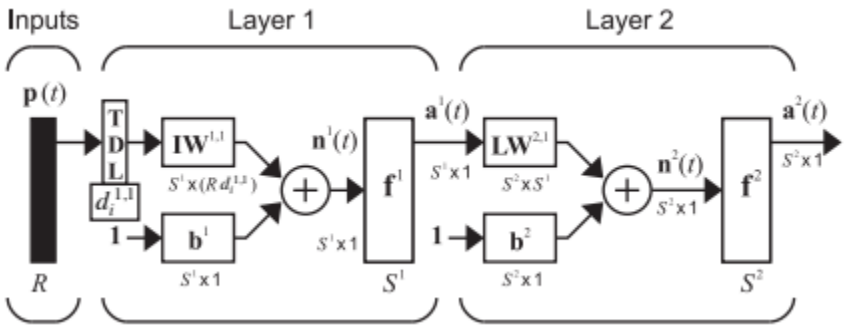

Figure 2. Textbook diagram of a focused time delay neural network (FTDNN). The tapped delay line is placed at the input of the network.

4.2 NARX

The NARX model augments the delayed exogenous input history with a delayed history of the target series itself. In other words, the network sees both external drivers and recent PV-power output values. This makes NARX a natural model for dynamic-system forecasting because the predicted series often depends on its own recent past. As with FTDNN, both linear and nonlinear versions were tested.

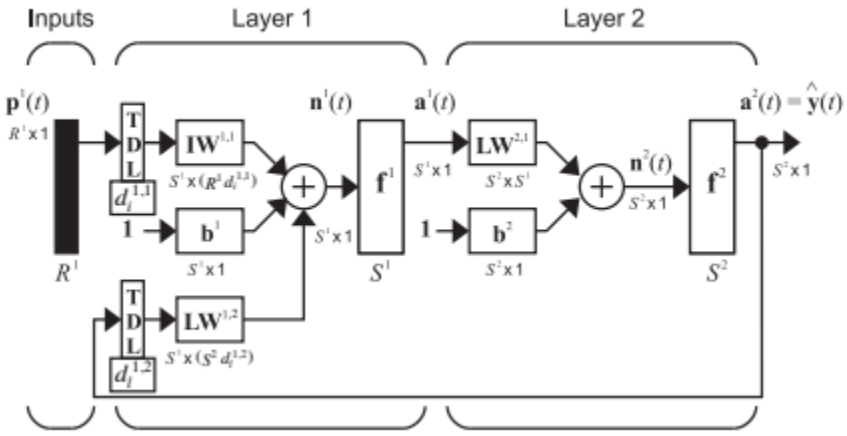

Figure 3. Textbook diagram of a NARX network. The model combines external inputs with autoregressive feedback from past outputs.

4.3 LSTM

The LSTM implementation uses a one-layer recurrent network with hidden size 48 and a small fully connected output head. Unlike the flattened FTDNN and NARX inputs, the LSTM receives the full time sequence of exogenous features plus the target channel. The recurrent architecture uses gated memory to control what information enters, remains in, and exits the memory state.

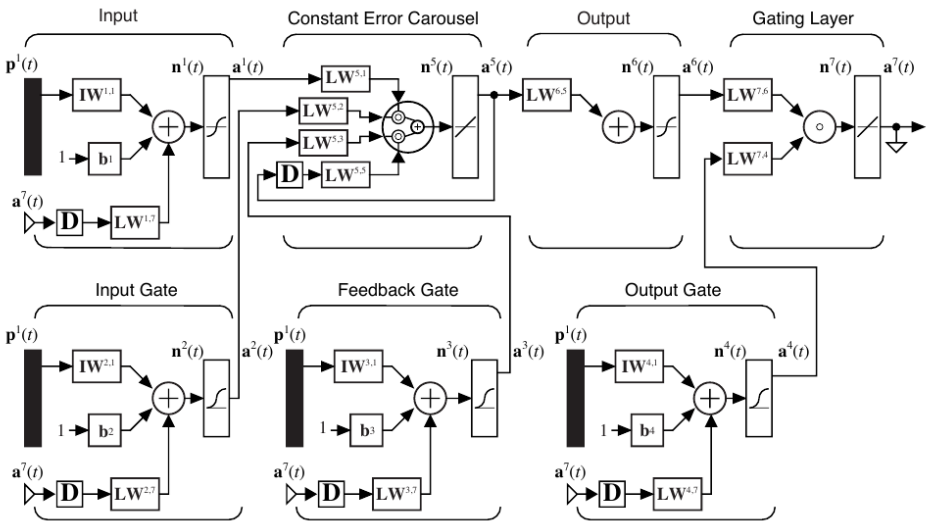

Figure 4. Textbook diagram of the complete LSTM architecture. Input, forget, and output gates surround a recurrent memory path.

4.4 Training objective and evaluation equations

All learned models were trained as residual predictors over persistence. Let y_t denote the true PV power at the target time and let p_t denote the persistence estimate, which is simply the most recent observed PV power carried forward to the forecast horizon. The model predicts a residual $r_t = y_t - p_t$, and the final forecast becomes $\hat{y}_t = p_t + \hat{r}_t$. This approach centers the learning problem around the part of the signal that the persistence baseline fails to capture.

$$r_t = y_t - p_t \qquad and \qquad \hat{y}_t = p_t + \hat{r}_t$$

$$MAE = (1/n)\ \Sigma\ |y_t - \hat{y}_t|$$

$$RMSE = sqrt[(1/n)\ \Sigma\ (y_t - \hat{y}_t)^2]$$

$$R^2 = 1 - [\Sigma\ (y_t - \hat{y}_t)\text{^}2] / [\Sigma\ (y_t - \bar{y})^2]$$

Optimization uses mean squared error on the scaled residual target, the AdamW optimizer, a ReduceLROnPlateau learning-rate scheduler, and early stopping based on validation loss. This combination was chosen to provide stable training while limiting overfitting.

5. Experimental Setup

The implementation is written in PyTorch. The script reads all daily logs from a logs folder, parses and sorts timestamps, converts numeric columns, and aligns the data to a one-minute grid using a backward merge-as-of operation with a maximum two-minute tolerance. Rows with missing target values or missing required features are removed. This prevents long outages from appearing to be continuous observations.

The final run used a 120-minute lookback window and a +60-minute forecast horizon. The preprocessed time series was then split chronologically into 19,721 training rows, 4,226 validation rows, and 4,226 test

rows. Chronological splitting was necessary because random shuffling would leak future information into the training process.

- Windowing: sliding windows were built strictly from past observations.
- Input scaling: means and standard deviations were fitted on the training set only, then applied to validation and test sets.
- Batch size: 256.
- Learning rate: 1e-3.
- Early-stopping patience: 8 epochs.
- FTDNN nonlinear architecture: hidden sizes [128, 64] with ReLU and dropout.
- NARX nonlinear architecture: hidden sizes [128, 64] with ReLU and dropout.
- LSTM architecture: one recurrent layer, hidden size 48, dropout 0.10 in the output head.
- Data augmentation: none. Because this is a real-valued time-series regression problem, the study relied on chronological validation and early stopping rather than synthetic augmentation.

The data loader used standard PyTorch Dataset and DataLoader classes. FlatDataset served the flattened FTDNN and NARX inputs, while SequenceDataset served the 3D sequence input required by the LSTM. At evaluation time, every learned model was compared against the persistence baseline on the same test timeline.

6. Results and Discussion

The persistence baseline for the final run had MAE = 35.452 W and RMSE = 50.638 W. Every learned model improved on this baseline, but the strongest overall performance came from the nonlinear FTDNN. Table 1 summarizes the final comparison.

Model	MAE (W)	RMSE (W)	R^2	MAE Improvement vs. Persistence (W)
ftdnn_nonlinear	22.449	31.218	0.5452	13.003
narx_nonlinear	23.018	32.856	0.4962	12.434
lstm	25.541	33.380	0.4800	9.911
narx_linear	28.790	37.510	0.3434	6.662
ftdnn_linear	30.547	39.566	0.2694	4.905

Table 1. Final model comparison on the +60 minute forecasting task.

The nonlinear FTDNN achieved the best final performance, with MAE = 22.449 W, RMSE = 31.218 W, and R^2 = 0.5452. The nonlinear NARX model ranked second, and the LSTM ranked third. Both linear baselines were clearly worse than their nonlinear counterparts.

This ranking is important because it shows that nonlinearity mattered more than raw architectural complexity. The linear FTDNN was the weakest model, yet the nonlinear FTDNN was the best model overall. In other words, delayed inputs alone were not enough; the network needed nonlinear layers to model interactions among irradiance, battery state, and recent power history.

The nonlinear NARX remained competitive because past output history adds useful information for a dynamic system. However, the extra autoregressive structure was not enough to outperform the simpler nonlinear FTDNN in this particular one-hour horizon setting.

The LSTM result is also meaningful even though it did not win. LSTM outperformed both linear baselines and remained competitive with the nonlinear feedforward approaches. Still, its final error was higher than the nonlinear FTDNN and nonlinear NARX. This suggests that for a 60-minute horizon, the local finite-memory structure in the recent window may already capture most of the useful predictive information.

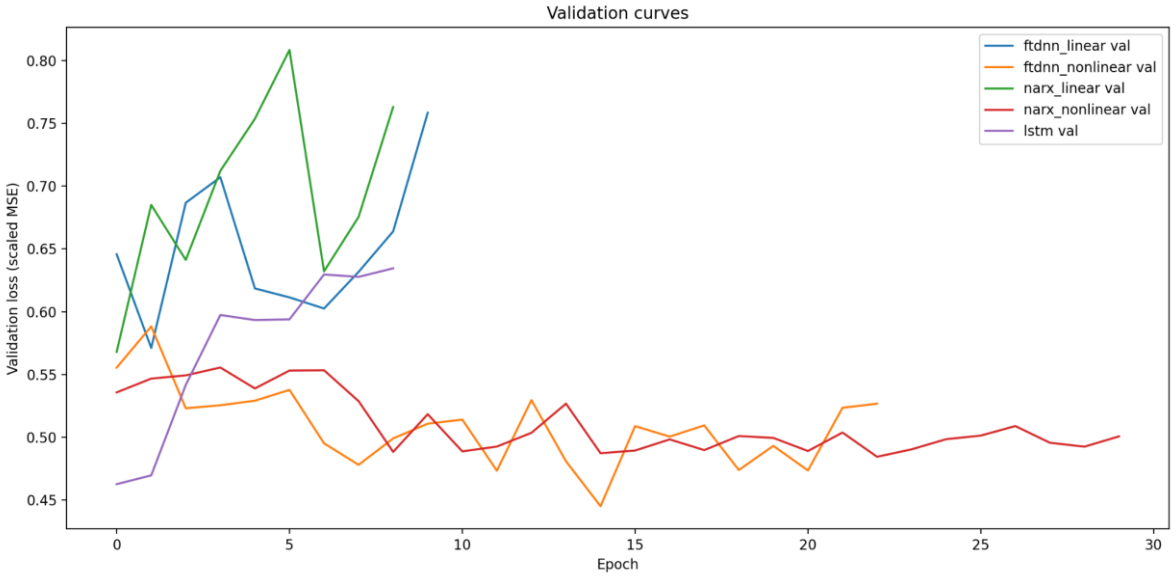

Figure 5. Validation-loss curves from the final run. The nonlinear FTDNN and nonlinear NARX settled to lower validation losses than the linear variants.

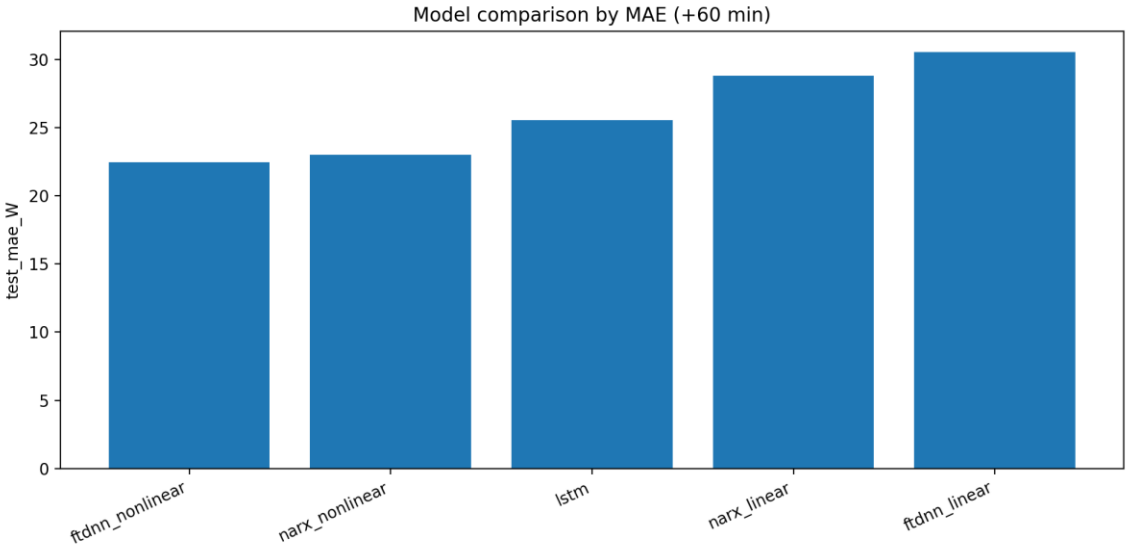

Figure 6. MAE comparison across all five learned models. Lower values indicate smaller average error in watts.

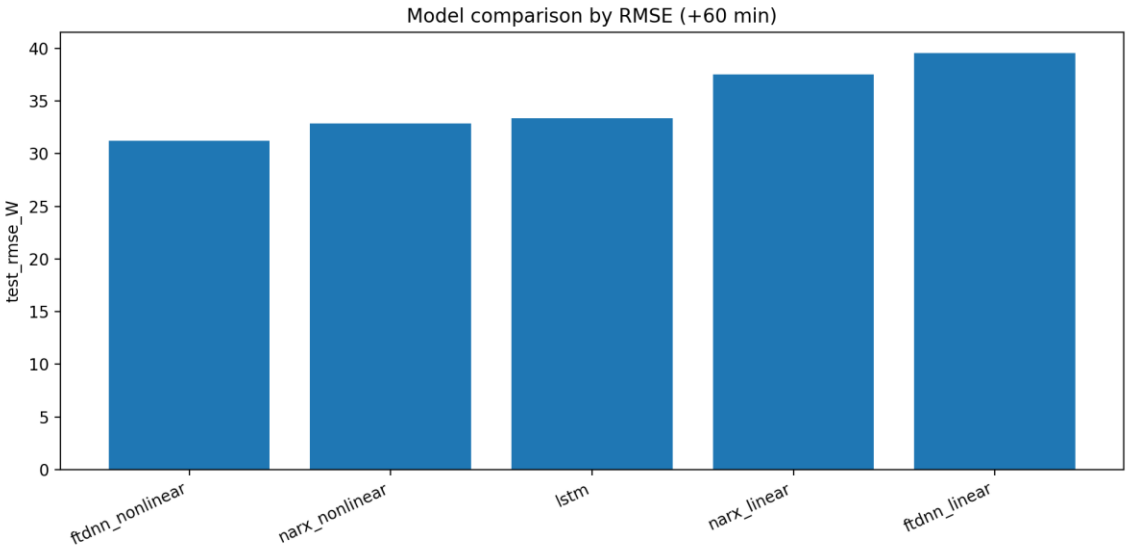

Figure 7. RMSE comparison across all five learned models. The same ranking seen in MAE holds under a stricter error metric.

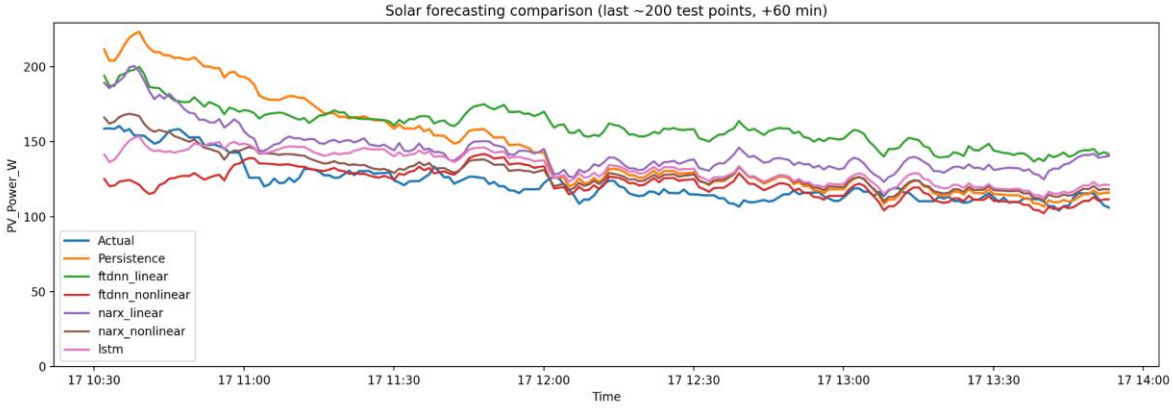

Figure 8. Actual vs. predicted PV power on the final portion of the test set. The nonlinear FTDNN and nonlinear NARX generally remain closest to the actual series.

7. Summary and Conclusions

This project began as an LSTM forecasting study but became a stronger comparative analysis after the inclusion of FTDNN and NARX baselines. The final results show that the best-performing model for this dataset and forecasting horizon was the nonlinear FTDNN.

Three conclusions stand out. First, the best overall result came from the simpler nonlinear delayed-input model, not the recurrent benchmark. Second, nonlinearity mattered greatly: for both FTDNN and NARX, the nonlinear versions clearly outperformed the linear baselines. Third, a more complex model was not automatically better. LSTM was competitive and meaningful, but it did not beat the best simpler model on this one-hour forecasting problem.

Several improvements remain possible. Future work should evaluate additional forecast horizons, test direct multi-step forecasting, continue tuning the LSTM or compare it with GRU, and extend the data record to include more seasonal coverage. Those additions would make the comparison more comprehensive and could further improve forecasting accuracy.

8. References

1. Martin Hagan, “Deep Learning Final Project,” course project brief, March 11, 2026.
2. Project proposal: “Short-Term Solar Power Forecasting Using Deep Learning,” submitted for the course proposal milestone.
3. Martin Hagan, “Nonlinear Sequence Processing,” Chapter 12 of the course textbook/chapter PDF. Figures for FTDNN, NARX, and LSTM were adapted from this chapter.
4. Project implementation: final_project_solar_compare.py, PyTorch-based training and evaluation script accompanying this report.
5. Generated project figures: sample_day_profile.png, training_curves.png, metric_mae.png, metric_rmse.png, and comparison_last200.png from the final run.

Appendix A. Code Organization

The main submitted code file is final_project_solar_compare.py. Its structure is organized into clearly labeled blocks so that the complete workflow is easy to review and reproduce.

- Imports and constants
- Small helpers
- Data loading and preprocessing
- Split and window builders
- Dataset classes for flat and sequence inputs
- Model definitions for MLP-based FTDNN/NARX and recurrent LSTM
- Training and evaluation utilities
- Plot helpers
- Main orchestration block for the end-to-end experiment

The script reads daily logs, constructs windows for each model family, trains the models, computes residual forecasts over persistence, writes model weights and metadata bundles, and saves the figures and metrics used throughout this report. Full documented code listings are included with the project repository submission.